Linked List Templates

Four patterns cover almost every linked list problem.
Consistent naming throughout: `sentinel`, `previous`, `current`, `next`, `slow`, `fast`.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
203	Remove Linked List Elements	Remove all nodes whose value equals `val`.		O(n)	O(1)	Standard
83	Remove Duplicates from Sorted List	Keep exactly one copy of each value. List is sorted.				Standard
82	Remove Duplicates from Sorted List II	Remove ALL nodes that have duplicate values — only distinct-valued nodes remain.		O(n)	O(1)	Standard
19	Remove Nth Node From End of List	Remove the nth node from the end. Return the head.				Standard
206	Reverse Linked List	Reverse the entire linked list. Return the new head.	flip each node's arrow to point at the node you just came from; `previous` is the reversed list built so far.			Standard
92	Reverse Linked List II	Reverse only the nodes from position `left` to `right` (1-indexed).				Standard
25	Reverse Nodes in k-Group	Reverse every consecutive group of k nodes. Leave remaining nodes (< k) as-is.		O(n)	O(1)	Standard
876	Middle of Linked List	Return the middle node. For even length, return the second middle.		O(n)	O(1)	Standard
141	Linked List Cycle	Return true if the list contains a cycle.		O(n)	O(1)	Standard
142	Linked List Cycle II	Return the node where the cycle begins. Return null if no cycle.	the distance from head to the cycle entry equals the distance from the meeting point to the entry, so two 1-step walkers from those spots collide exactly at the entry.	O(n)	O(1)	Standard
21	Merge Two Sorted Lists	Merge two sorted linked lists into one sorted list.	like a zipper — always splice on whichever current head is smaller, then advance that list.	O(n)	O(1)	Standard
23	Merge K Sorted Lists	Merge k sorted linked lists into one sorted list.		O(n log k)	O(k)	Standard
143	Reorder List	Reorder list as L0 → Ln → L1 → Ln-1 → L2 → ...				Standard
328	Odd Even Linked List	Group all odd-indexed nodes first, then even-indexed nodes. Preserve relative order within each group. (Indexing is 1-based.)				Standard
2	Add Two Numbers	Two non-empty linked lists represent non-negative integers stored in reverse order. Add and return the		O(max(m, n))	O(max(m, n))	loop until carry clears

#	Name	Description	Intuition	Time	Space	Key Implementation
		sum as a linked list in reverse order.				
88	Merge Sorted Array	Merge `nums2` into `nums1` in-place. `nums1` has enough space at the end. Both arrays are sorted.		O(m + n)	O(1)	three pointers from END
234	Palindrome Linked List	Check if a singly linked list is a palindrome in O(n) time and O(1) space.		O(n)	O(1)	in-place reverse
160	Intersection of Two Linked Lists	Find the node at which two singly linked lists intersect. Return null if they do not intersect.		O(m + n)	O(1)	redirect to other list on null
24	Swap Nodes in Pairs	Swap every two adjacent nodes in a linked list and return the modified head.	A sentinel removes head special-casing. For each pair, `previous` is the node before the pair; re-wire the three pointers so the second node leads, then advance `previous` two nodes forward.	O(n)	O(1)	Standard
61	Rotate List	Rotate a linked list to the right by `k` places.	Rotating right by `k` is equivalent to making the list circular, then breaking it `k % length` nodes before the old tail. Compute the length, close the ring, then walk to the new tail.	O(n)	O(1)	only the remainder matters
86	Partition List	Partition a linked list so all nodes with value `< x` come before nodes with value `>= x`. Preserve the relative order within each group.	Build two separate chains with two sentinels — one for the "less" group and one for the "greater-or-equal" group — then stitch them together. Preserving original order falls out naturally because nodes are appended in traversal order.	O(n)	O(1)	Standard
109	Convert Sorted List to Binary Search Tree	Convert a sorted linked list to a height-balanced BST.	A sorted list is an in-order traversal of a BST. Repeatedly pick the middle node as the subtree root so each side gets a balanced number of nodes; recurse on the left half (before middle) and right half (after middle).	O(n log n)	O(log n)	stop at exclusive tail
138	Copy List with Random Pointer	Deep-copy a linked list where each node has a `random` pointer to any node or null.	Interleave each clone directly after its original so a clone's `random` is reachable as `original.random.next` — no hash map needed. Three passes: weave clones in, wire random pointers, then unweave.	O(n)	O(1)	Standard
148	Sort List	Sort a linked list. Aim for O(n log n) time and O(1) extra space (top-down recursion uses O(log n) stack).	Merge sort fits linked lists perfectly — splitting is a pointer cut and merging reuses the #21 zipper. Find the middle, sort each half, merge.	O(n log n)	O(log n)	Standard
237	Delete Node in a Linked List	Delete a node from a singly linked list given only a reference to that node (not the head). The node is guaranteed not to be the tail.	Without access to the previous node you cannot unlink the given node directly, but you can copy the next node's value into it and then delete the next node instead.	O(1)	O(1)	copy successor's value
369	Plus One Linked List	A non-negative integer is stored as a linked list of digits in big-endian order	Reverse the list so addition flows least-significant-first like normal carry arithmetic, add one with carry	O(n)	O(1)	seed carry with the +1

#	Name	Description	Intuition	Time	Space	Key Implementation
		(most significant first). Add one to it and return the resulting list.	propagation, then reverse back. A possible new leading digit is handled by extending the tail after reversal.			
430	Flatten a Multilevel Doubly Linked List	Flatten a multilevel doubly linked list where nodes may have a `child` list. Produce a single-level list using the `next` pointers; all `child` pointers must be null.	A child list should be spliced in immediately after its parent, before the parent's original `next`. A stack lets you remember each deferred `next` while you dive into a child branch — effectively an iterative DFS.	O(n)	O(n)	clear child after splicing
445	Add Two Numbers II	Two numbers are stored as linked lists in big-endian order (most significant digit first). Add them and return the sum as a linked list, also in big-endian order.	Addition needs least-significant digits first, but reversing the inputs is discouraged here. Push all digits onto two stacks; popping yields least-significant-first. Build the result by prepending nodes so the final list is big-endian.	O(m + n)	O(m + n)	prepend to keep big-endian order
708	Insert into a Sorted Circular Linked List	Insert a value into a sorted circular linked list and return a reference to any node in the list. The given pointer may reference any node, or be null for an empty list.	Walk one full loop looking for the correct gap between a node and its successor. Three cases cover it: a normal in-between slot, the wrap-around boundary (max→min) where the value is an extreme, and a uniform-value list where any spot works.	O(n)	O(1)	wrap-around boundary
725	Split Linked List in Parts	Split a linked list into `k` consecutive parts as equal in size as possible. Earlier parts have sizes greater than or equal to later parts; some trailing parts may be empty.	With length `n`, each part gets `n / k` nodes, and the first `n % k` parts get one extra. Walk through cutting off each part at its computed size.	O(n)	O(k)	first `remainder` parts get +1
1171	Remove Zero Sum Consecutive Nodes from Linked List	Repeatedly remove consecutive sequences of nodes that sum to zero until no such sequence remains. Return the resulting list.	Use prefix sums. If two nodes share the same running prefix sum, every node strictly between them sums to zero and can be dropped. A hash map from prefix sum to node lets you splice out those runs in one pass.	O(n)	O(n)	keep the latest node per sum
1265	Print Immutable Linked List in Reverse	Print all values of an immutable linked list in reverse, using only the exposed `getNext()` and `printValue()` API, without modifying the list.	Recursion naturally reverses order — recurse to the end first, then print on the way back up the call stack.	O(n)	O(n)	recurse first, print on unwind

When to Use — Signal → Pattern

Signal in the prompt	Pattern
"remove / delete / skip" nodes by value or duplicate	Delete / Filter (sentinel + `previous`)
"reverse" the list, a sub-range, or in k-groups	Reversal
"middle node", "cycle", "nth node from the end"	Fast / Slow
"merge two sorted lists" (or k lists)	Merge (sentinel + heap for k)
chained steps ("reorder", "palindrome", "sort")	Composite (combine the above)

All Four Templates

Variables: `sentinel` = dummy node before head · `previous` = last kept node · `current` = node under inspection · `next` = saved successor · `slow` / `fast` = 1×/2× walkers · `a` / `b` = the two input lists **Pseudocode:**

```
# 1. DELETE / FILTER
make sentinel pointing at head; previous = sentinel, current = head
while current exists:
    if current should be deleted: rewire previous.next to skip current
    else: advance previous to current
    advance current
return sentinel.next
# 2. REVERSAL
previous = null, current = head
while current exists:
    save next = current.next
    point current backward at previous
    advance previous to current, current to next
return previous as new head
# 3. FAST / SLOW
slow = head, fast = head
while fast and fast.next exist:
    advance slow one step, fast two steps
# slow is at middle (or slow == fast means cycle)
# 4. MERGE
make sentinel; current = sentinel
while both a and b exist:
    splice the smaller head onto current; advance that list
    advance current
attach whichever list remains
return sentinel.next
```

```
// 1. DELETE / FILTER  — sentinel guards head removal; previous tracks last kept node
// previous always points at the last node you decided to keep, so re-wiring it skips anything unwanted
ListNode sentinel = new ListNode(0);
sentinel.next = head;
ListNode previous = sentinel, current = head;
while (current != null) {
    if (shouldDelete(current)) {
        previous.next = current.next;   // skip node
    } else {
        previous = current;             // keep node, advance previous
    }
    current = current.next;
}
return sentinel.next;

// 2. REVERSAL — re-wire one node at a time
// flip each arrow to point backward; previous is the growing reversed list trailing behind current.  —
ListNode previous = null, current = head;
while (current != null) {
    ListNode next = current.next;   // save before overwrite
    current.next = previous;        // reverse pointer
    previous = current;             // advance
    current = next;
}
return previous;  // new head

// 3. FAST / SLOW — two pointers at different speeds
// fast covers twice the distance, so when it hits the end slow is exactly halfway; in a loop they must
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// slow is at middle  (or cycle detection: slow == fast → cycle found)

// 4. MERGE — sentinel collects nodes from two lists in order
// repeatedly splice the smaller head onto a growing result tail, like a zipper.  — WHEN: "merge two (o
ListNode sentinel = new ListNode(0);
ListNode current = sentinel;
while (a != null && b != null) {
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}
current.next = (a != null) ? a : b;
return sentinel.next;
```

Pattern 1 — Delete / Filter

`previous` stays on the last **kept** node. When deleting, re-wire `previous.next` to skip `current`. When keeping, advance `previous` to `current`. Always advance `current`.

#203 Remove Linked List Elements

Description: Remove all nodes whose value equals `val`.
Delete condition: `current.val == val`

Variables: `sentinel` = dummy before head · `previous` = last kept node · `current` = node under inspection · `val` = value to remove **Pseudocode:**

```
make sentinel pointing at head; previous = sentinel, current = head
while current exists:
    if current.val == val: rewire previous.next to skip current
    else: advance previous to current
    advance current
return sentinel.next
```

```java
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (current.val == val) {
                previous.next = current.next;
            } else {
                previous = current;
            }
            current = current.next;
        }
        return sentinel.next;
    }
}
```

#83 Remove Duplicates from Sorted List

Description: Keep exactly one copy of each value. List is sorted.
Delete condition: `previous != sentinel && previous.val == current.val`

Variables: `sentinel` = dummy before head · `previous` = last kept node · `current` = node under inspection
Pseudocode:

```
make sentinel pointing at head; previous = sentinel, current = head
while current exists:
    if previous is a real node with same value as current: skip current (drop the duplicate)
    else: advance previous to current
    advance current
return sentinel.next
```

```java
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (previous != sentinel && previous.val == current.val) {
                previous.next = current.next;   // skip duplicate
            } else {
                previous = current;
            }
            current = current.next;
        }
        return sentinel.next;
    }
}
```

#82 Remove Duplicates from Sorted List II

Description: Remove ALL nodes that have duplicate values — only distinct-valued nodes remain.
Key difference from #83: when a duplicate run is detected, skip the entire run (including the first occurrence). Inner `while` advances `current` past the whole run; `previous.next` is rewired to skip all of them.

Variables: `sentinel` = dummy before head · `previous` = last kept node · `current` = node under inspection · `dupVal` = value of the run being removed **Pseudocode:**

```
make sentinel pointing at head; previous = sentinel, current = head
while current exists:
    if current starts a duplicate run (current.val == next.val):
        record dupVal; advance current past every node equal to dupVal
        rewire previous.next to skip the whole run
    else:
        advance previous to current; advance current
return sentinel.next
```

```java
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (current.next != null && current.val == current.next.val) {
                int dupVal = current.val;
                while (current != null && current.val == dupVal)
                    current = current.next;       // advance past entire run
                previous.next = current;          // skip all duplicates
            } else {
                previous = current;
                current = current.next;
            }
        }
        return sentinel.next;
    }
}
```

#19 Remove Nth Node From End of List

Description: Remove the nth node from the end. Return the head.
Key: `fast` starts n+1 steps ahead of `slow` (both from sentinel). When `fast == null`, `slow` is just before the target node.

Variables: `sentinel` = dummy before head · `fast` = lead pointer · `slow` = trailing pointer (lands just before target) · `n` = offset from the end **Pseudocode:**

```
make sentinel pointing at head; fast = sentinel, slow = sentinel
move fast forward n+1 steps
while fast exists: advance slow and fast together
# slow now sits just before the node to delete
rewire slow.next to skip the target
return sentinel.next
```

```java
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode fast = sentinel, slow = sentinel;
        for (int i = 0; i <= n; i++) fast = fast.next;   // n+1 steps ahead
        while (fast != null) { slow = slow.next; fast = fast.next; }
        slow.next = slow.next.next;                      // delete target
        return sentinel.next;
    }
}
```

Pattern 2 — Reversal

#206 Reverse Linked List

Description: Reverse the entire linked list. Return the new head.
Key: re-wire one node per iteration. `previous` trails behind as the new "next" for each node.

Intuition: flip each node's arrow to point at the node you just came from; `previous` is the reversed list built so far.

Variables: `previous` = reversed list built so far · `current` = node being re-wired · `next` = saved successor
Pseudocode:

```
previous = null, current = head
while current exists:
    save next = current.next
    point current backward at previous
    advance previous to current, current to next
return previous as new head
```

```java
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode previous = null, current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        return previous;
    }
}
```

#92 Reverse Linked List II

Description: Reverse only the nodes from position `left` to `right` (1-indexed).
Key: walk `previous` to the node just before `left`. Then insert-at-front `(right - left)` times — each time take `current.next`, unlink it, and attach it right after `previous`.

Variables: `sentinel` = dummy before head · `previous` = node just before the reversal region · `current` = first node of the region (stays put, drifts to the tail) · `next` = node being moved to the front · `left` / `right` = 1-indexed bounds
Pseudocode:

```
make sentinel pointing at head; previous = sentinel
walk previous forward left-1 steps (to node before position left)
current = previous.next
repeat right-left times:
    next = current.next              # node just after current
    unlink next from current
    splice next in right after previous (front of reversed region)
return sentinel.next
```

```java
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel;
        for (int i = 0; i < left - 1; i++) previous = previous.next;
        ListNode current = previous.next;
        for (int i = 0; i < right - left; i++) {
            ListNode next = current.next;
            current.next = next.next;     // unlink next
            next.next = previous.next;    // attach next at front of reversed region
            previous.next = next;
        }
        return sentinel.next;
    }
}
```

#25 Reverse Nodes in k-Group

Description: Reverse every consecutive group of k nodes. Leave remaining nodes (< k) as-is.
Key: `groupPrev` is the tail of the already-processed part. Find the kth node ahead; if it exists, reverse the group using the same insert-at-front technique as #92.

Variables: `sentinel` = dummy before head · `groupPrev` = tail of the processed part (node before current group) · `kth` = kth node ahead (end of current group) · `groupNext` = node after the group · `previous` / `current` / `next` = reversal pointers · `tmp` = old group head (becomes new tail) **Pseudocode:**

```
make sentinel pointing at head; groupPrev = sentinel
loop:
    kth = node k steps past groupPrev; if none, stop (leftover < k stays as-is)
    groupNext = kth.next
    reverse the k nodes between groupPrev.next and groupNext (insert-at-front style)
    tmp = old first node of group (now its tail)
    link groupPrev.next to kth (new head of reversed group)
    advance groupPrev to tmp
return sentinel.next
# getKth: walk k steps from node, return where you land (or null)
```

```java
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode groupPrev = sentinel;
        while (true) {
            ListNode kth = getKth(groupPrev, k);
            if (kth == null) break;
            ListNode groupNext = kth.next;
            ListNode previous = groupNext, current = groupPrev.next;
            while (current != groupNext) {
                ListNode next = current.next;
                current.next = previous;
                previous = current;
                current = next;
            }
            ListNode tmp = groupPrev.next;   // will become new tail of this group
            groupPrev.next = kth;            // kth is new head of reversed group
            groupPrev = tmp;
        }
        return sentinel.next;
    }
    private ListNode getKth(ListNode node, int k) {
        while (node != null && k-- > 0) node = node.next;
        return node;
    }
}
```

Pattern 3 — Fast / Slow Pointers

`fast` moves 2× per step. When `fast` reaches the end, `slow` is at the middle.
When `slow == fast` after the start, a cycle is detected.

#876 Middle of Linked List

Description: Return the middle node. For even length, return the second middle.

Variables: `slow` = 1×-speed pointer (lands on middle) · `fast` = 2×-speed pointer **Pseudocode:**

```
slow = head, fast = head
while fast and fast.next exist:
    advance slow one step, fast two steps
return slow as the middle
```

```java
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

#141 Linked List Cycle

Description: Return true if the list contains a cycle.

Variables: `slow` = 1×-speed pointer · `fast` = 2×-speed pointer **Pseudocode:**

```
slow = head, fast = head
while fast and fast.next exist:
    advance slow one step, fast two steps
    if slow == fast: a cycle exists, return true
return false (fast reached the end)
```

```java
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

#142 Linked List Cycle II

Description: Return the node where the cycle begins. Return null if no cycle.
Key: after slow meets fast inside the cycle, reset `slow` to `head`. Both then move 1 step at a time — they meet at the cycle entry.

Intuition: the distance from head to the cycle entry equals the distance from the meeting point to the entry, so two 1-step walkers from those spots collide exactly at the entry.

Variables: `slow` = 1×-speed pointer (reset to head after meeting) · `fast` = 2×-speed pointer **Pseudocode:**

```
slow = head, fast = head
while fast and fast.next exist:
    advance slow one step, fast two steps
    if slow == fast:                # meeting point inside the cycle
        reset slow to head
        advance slow and fast one step each until they meet
        return that node as the cycle entry
return null (no cycle)
```

```java
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                slow = head;                            // reset slow to head
                while (slow != fast) {
                    slow = slow.next;
                    fast = fast.next;                   // both move 1 step
                }
                return slow;                            // cycle entry
            }
        }
        return null;
    }
}
```

Pattern 4 — Merge

`sentinel` collects the merged result. `current` is the write pointer.

#21 Merge Two Sorted Lists

Description: Merge two sorted linked lists into one sorted list.

Intuition: like a zipper — always splice on whichever current head is smaller, then advance that list.

Variables: `sentinel` = dummy result head · `current` = write pointer (tail of merged list) · `list1` / `list2` = remaining heads of the two inputs **Pseudocode:**

```
make sentinel; current = sentinel
while both lists exist:
    splice the smaller of list1/list2 onto current; advance that list
    advance current
attach whichever list still has nodes
return sentinel.next
```

```java
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                current.next = list1;
                list1 = list1.next;
            } else {
                current.next = list2;
                list2 = list2.next;
            }
            current = current.next;
        }
        current.next = (list1 != null) ? list1 : list2;
        return sentinel.next;
    }
}
```

#23 Merge K Sorted Lists

Description: Merge k sorted linked lists into one sorted list.
Key: min-heap of size ≤ k. Always extract the smallest current head; push its next node back.

Variables: `sentinel` = dummy result head · `current` = write pointer · `pq` = min-heap of current list heads keyed by value **Pseudocode:**

```
make sentinel; current = sentinel
push every non-null list head into a min-heap
while heap not empty:
    pop the smallest node; append it to current; advance current
    if that node has a next, push the next onto the heap
return sentinel.next
```

```java
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode node : lists) if (node != null) pq.offer(node);
        while (!pq.isEmpty()) {
            current.next = pq.poll();
            current = current.next;
            if (current.next != null) pq.offer(current.next);
        }
        return sentinel.next;
    }
}
```

Pattern 5 — Composite (Multi-step)

Problems that chain multiple patterns together.

#143 Reorder List

Description: Reorder list as L0 → Ln → L1 → Ln-1 → L2 → ...
Steps: (1) find middle with slow/fast, (2) reverse second half, (3) merge the two halves.

Variables: `slow` / `fast` = middle-finding pointers · `second` = head of the second half · `previous` / `current` / `next` = reversal pointers · `first` = walker over first half · `nextFirst` / `nextSecond` = saved successors during the zip
Pseudocode:

```
# Step 1: find middle (fast starts one ahead so first half >= second)
slow = head, fast = head.next
advance slow 1× and fast 2× until fast reaches the end
# Step 2: cut and reverse the second half
second = slow.next; cut list at slow
reverse second into previous
# Step 3: zip first half and reversed second half alternately
first = head, second = previous
while second exists:
    save nextFirst, nextSecond
    splice second right after first
    advance first to nextFirst, second to nextSecond
```

```java
class Solution {
    public void reorderList(ListNode head) {
        // Step 1: find middle
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // Step 2: reverse second half
        ListNode second = slow.next;
        slow.next = null;
        ListNode previous = null, current = second;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        // Step 3: merge first half and reversed second half
        ListNode first = head;
        second = previous;
        while (second != null) {
            ListNode nextFirst = first.next, nextSecond = second.next;
            first.next = second;
            second.next = nextFirst;
            first = nextFirst;
            second = nextSecond;
        }
    }
}
```

#328 Odd Even Linked List

Description: Group all odd-indexed nodes first, then even-indexed nodes. Preserve relative order within each group. (Indexing is 1-based.)

Variables: `odd` = tail of the odd-index chain · `even` = tail of the even-index chain · `evenHead` = saved head of the even chain (to splice on at the end) **Pseudocode:**

```
if list empty: return null
odd = head, even = head.next, evenHead = even
while even and even.next exist:
    link odd to the node after even; advance odd
    link even to the node after odd; advance even
attach evenHead after the odd tail
return head
```

```java
class Solution {
    public ListNode oddEvenList(ListNode head) {
        if (head == null) return null;
        ListNode odd = head, even = head.next, evenHead = even;
        while (even != null && even.next != null) {
            odd.next = even.next;   odd = odd.next;
            even.next = odd.next;   even = even.next;
        }
        odd.next = evenHead;
        return head;
    }
}
```

Pattern Summary

Pattern	Sentinel	Naming	When to use
Delete / Filter	Yes	`previous` , `current`	Remove or skip nodes by condition
Reversal	No (Yes for partial)	`previous` , `current` , `next`	Re-wire pointer direction
Fast / Slow	No	`slow` , `fast`	Middle, cycle, nth from end
Merge	Yes	`current`	Combine two or more lists
Composite	Both	Mix of above	Reorder, sort, rearrange

Sentinel Cheat Sheet

SENTINEL: dummy node before head so head removal needs no special-casing; use it whenever the head itself might be removed/replaced.

Variables: `sentinel` = dummy node placed before head; `sentinel.next` always tracks the real head **Pseudocode:**

```
make sentinel; point sentinel.next at head
do the operations (head may get removed or replaced)
return sentinel.next as the true head
```

```
// Always use sentinel when the head itself might be removed or replaced:
ListNode sentinel = new ListNode(0);
sentinel.next = head;
// ... operations ...
return sentinel.next;   // head may have changed; sentinel.next is always correct
```

82 vs 83 — Side by Side

```
                      #83 (keep one)              #82 (remove all)
Delete condition:     previous.val == current.val  current.val == current.next.val
Action on match:      skip current only            inner while to skip entire run
previous advances?:   only when keeping            only when not a dup
```

#2 Add Two Numbers

Description: Two non-empty linked lists represent non-negative integers stored in reverse order. Add and return the sum as a linked list in reverse order.

Algorithm: Use sentinel + carry variable. Traverse both lists simultaneously; at each step compute `sum = l1.val + l2.val + carry`. Create a new node with `sum % 10`, carry `sum / 10` forward. Continue while either list has nodes or carry is non-zero.

Variables: `sentinel` = dummy result head · `current` = write pointer · `carry` = carry-over digit · `sum` = per-step total · `l1` / `l2` = remaining input heads (reverse order) **Pseudocode:**

```
make sentinel; current = sentinel, carry = 0
while l1 or l2 has nodes, or carry is non-zero:
    sum = carry
    add l1's digit if present and advance l1
    add l2's digit if present and advance l2
    carry = sum / 10
    append a new node with digit sum % 10; advance current
return sentinel.next
```

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) {   // ← VARIATION: loop until carry clears
            int sum = carry;
            if (l1 != null) { sum += l1.val; l1 = l1.next; }
            if (l2 != null) { sum += l2.val; l2 = l2.next; }
            carry = sum / 10;                               // ← VARIATION: propagate carry
            current.next = new ListNode(sum % 10);
            current = current.next;
        }
        return sentinel.next;
    }
}
```

Time O(max(m, n)) | **Space** O(max(m, n))

#88 Merge Sorted Array

Description: Merge `nums2` into `nums1` in-place. `nums1` has enough space at the end. Both arrays are sorted.

Algorithm: Fill from the END backwards using three pointers `i` (end of nums1 values), `j` (end of nums2), `k` (end of merged array). This avoids overwriting unprocessed elements in nums1.

Variables: `i` = index of last real value in nums1 · `j` = index of last value in nums2 · `k` = write index at the very end of nums1 **Pseudocode:**

```
i = m−1, j = n−1, k = m+n−1
while both i and j are in range:
    write the larger of nums1[i]/nums2[j] into nums1[k]; step that source back; step k back
copy any remaining nums2 values into the front of nums1
# leftover nums1 values are already in place
```

```java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;          // ← VARIATION: three pointers from END
        while (i >= 0 && j >= 0)
            nums1[k--] = (nums1[i] >= nums2[j]) ? nums1[i--] : nums2[j--];
        while (j >= 0) nums1[k--] = nums2[j--];            // ← copy remaining nums2
    }
}
```

Time O(m + n) | **Space** O(1)

#234 Palindrome Linked List

Description: Check if a singly linked list is a palindrome in O(n) time and O(1) space.

Algorithm: Composite — (1) fast/slow to find middle, (2) reverse the second half in-place, (3) compare both halves. Variation: combines three separate patterns.

Variables: `slow` / `fast` = middle-finding pointers · `previous` / `current` / `next` = reversal pointers · `i` / `j` = comparison walkers over the two halves **Pseudocode:**

```
# Step 1: find middle
advance slow 1× and fast 2× until fast reaches the end
# Step 2: reverse from slow onward into previous
reverse the second half
# Step 3: compare halves
i = head, j = previous
while j exists:
    if values differ: return false
    advance i and j
return true
```

```java
class Solution {
    public boolean isPalindrome(ListNode head) {
        // Step 1: find middle using fast/slow
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // Step 2: reverse second half                    // ← VARIATION: in-place reverse
        ListNode previous = null, current = slow;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        // Step 3: compare halves
        ListNode i = head, j = previous;
        while (j != null) {
            if (i.val != j.val) return false;
            i = i.next;
            j = j.next;
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#160 Intersection of Two Linked Lists

Description: Find the node at which two singly linked lists intersect. Return null if they do not intersect.

Algorithm: Two pointers `a` and `b` start at `headA` and `headB`. When either reaches the end, redirect it to the other list's head. After at most `m + n` steps they meet at the intersection (or both become null for no intersection). This works because both pointers travel the same total distance.

Variables: `a` = walker starting at headA (jumps to headB on null) · `b` = walker starting at headB (jumps to headA on null) **Pseudocode:**

```
a = headA, b = headB
while a != b:
    advance a; if it fell off the end, redirect it to headB
    advance b; if it fell off the end, redirect it to headA
return a   # the intersection node, or null if none
```

```java
class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode a = headA, b = headB;
        while (a != b) {
            a = (a == null) ? headB : a.next;   // ← VARIATION: redirect to other list on null
            b = (b == null) ? headA : b.next;
        }
        return a;   // either the intersection node, or null
    }
}
```

Time O(m + n) | **Space** O(1)

Additional Reference Problems

#24 Swap Nodes in Pairs

Description: Swap every two adjacent nodes in a linked list and return the modified head.

Intuition: A sentinel removes head special-casing. For each pair, `previous` is the node before the pair; re-wire the three pointers so the second node leads, then advance `previous` two nodes forward.

Algorithm: Loop while there are at least two nodes ahead of `previous`. Name them `first` and `second`. Splice so order becomes `second → first`, reconnect `first.next` to whatever followed `second`, then move `previous` to `first` (now the tail of the swapped pair).

Variables: `sentinel` = dummy before head · `previous` = node before the current pair · `first` / `second` = the two nodes being swapped **Pseudocode:**

```
make sentinel pointing at head; previous = sentinel
while two nodes exist after previous:
    first = previous.next, second = first.next
    point first past the pair (to second.next)
    point second at first
    link previous to second (new pair head)
    advance previous to first (tail of swapped pair)
return sentinel.next
```

```java
class Solution {
    public ListNode swapPairs(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel;
        while (previous.next != null && previous.next.next != null) {
            ListNode first = previous.next;
            ListNode second = first.next;
            first.next = second.next;   // first now points past the pair
            second.next = first;        // second leads the pair
            previous.next = second;     // link prior node to new pair head
            previous = first;           // advance to tail of swapped pair
        }
        return sentinel.next;
    }
}
```

Time O(n) | **Space** O(1)

#61 Rotate List

Description: Rotate a linked list to the right by `k` places.

Intuition: Rotating right by `k` is equivalent to making the list circular, then breaking it `k % length` nodes before the old tail. Compute the length, close the ring, then walk to the new tail.

Algorithm: Measure length and find the tail. Connect tail to head to form a cycle. The new tail sits `length - (k % length)` steps from the head; the node after it is the new head. Break the cycle there.

Variables: `length` = node count · `tail` = last node · `k` = rotation amount (reduced mod length) · `stepsToNewTail` = distance from head to new tail · `current` = walker to the new tail · `newHead` = node after new tail **Pseudocode:**

```
if list empty/single or k==0: return head
walk to tail while counting length
k = k mod length; if k==0 return head (no-op rotation)
close the list into a ring (tail.next = head)
stepsToNewTail = length - k
walk current to the new tail
newHead = current.next
break the ring (current.next = null)
return newHead
```

```java
class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        if (head == null || head.next == null || k == 0) {
            return head;
        }
        int length = 1;
        ListNode tail = head;
        while (tail.next != null) {
            tail = tail.next;
            length++;
        }
        k = k % length;                 // ← VARIATION: only the remainder matters
        if (k == 0) {
            return head;
        }
        tail.next = head;               // close into a ring
        int stepsToNewTail = length - k;
        ListNode current = head;
        for (int i = 1; i < stepsToNewTail; i++) {
            current = current.next;
        }
        ListNode newHead = current.next;
        current.next = null;            // break the ring
        return newHead;
    }
}
```

Time O(n) | **Space** O(1)

#86 Partition List

Description: Partition a linked list so all nodes with value `< x` come before nodes with value `>= x`. Preserve the relative order within each group.

Intuition: Build two separate chains with two sentinels — one for the "less" group and one for the "greater-or-equal" group — then stitch them together. Preserving original order falls out naturally because nodes are appended in traversal order.

Algorithm: Walk `current` over the list, appending each node to the `less` tail or `greater` tail by comparing with `x`. Terminate the `greater` chain, then connect the `less` tail to the head of the `greater` chain.

Variables: `lessSentinel` / `greaterSentinel` = dummy heads of the two chains · `less` / `greater` = tails of each chain · `current` = walker over input · `x` = partition value **Pseudocode:**

```
make two sentinels (less chain, greater-or-equal chain); less = lessSentinel, greater = greaterSentinel
for each node current:
    if current.val < x: append to less tail
    else: append to greater tail
terminate the greater chain (greater.next = null)
splice less tail to the start of the greater chain
return lessSentinel.next
```

```java
class Solution {
    public ListNode partition(ListNode head, int x) {
        ListNode lessSentinel = new ListNode(0);
        ListNode greaterSentinel = new ListNode(0);
        ListNode less = lessSentinel, greater = greaterSentinel;
        ListNode current = head;
        while (current != null) {
            if (current.val < x) {
                less.next = current;
                less = less.next;
            } else {
                greater.next = current;
                greater = greater.next;
            }
            current = current.next;
        }
        greater.next = null;                    // terminate second chain
        less.next = greaterSentinel.next;       // splice less → greater
        return lessSentinel.next;
    }
}
```

Time O(n) | **Space** O(1)

#109 Convert Sorted List to Binary Search Tree

Description: Convert a sorted linked list to a height-balanced BST.

Intuition: A sorted list is an in-order traversal of a BST. Repeatedly pick the middle node as the subtree root so each side gets a balanced number of nodes; recurse on the left half (before middle) and right half (after middle).

Algorithm: Use fast/slow to find the middle of the current range `[head, tail)`. The middle becomes the root. Recurse on `[head, middle)` for the left child and `[middle.next, tail)` for the right child. The half-open range avoids cutting the list and keeps recursion clean.

Variables: `head` = start of current range · `tail` = exclusive end of range · `slow` / `fast` = middle-finding pointers within `[head, tail)` · `root` = subtree root built from the middle **Pseudocode:**

```
build(head, tail):
    if head == tail: return null (empty range)
    find middle of [head, tail) with slow 1× / fast 2× (fast stops at tail)
    root = node from slow's value
    root.left  = build(head, slow)        # left half before middle
    root.right = build(slow.next, tail)    # right half after middle
    return root
```

```java
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        return build(head, null);
    }
    private TreeNode build(ListNode head, ListNode tail) {
        if (head == tail) {
            return null;
        }
        ListNode slow = head, fast = head;
        while (fast != tail && fast.next != tail) {   // ← VARIATION: stop at exclusive tail
            slow = slow.next;
            fast = fast.next.next;
        }
        TreeNode root = new TreeNode(slow.val);
        root.left = build(head, slow);
        root.right = build(slow.next, tail);
        return root;
    }
}
```

Time O(n log n) | **Space** O(log n)

#138 Copy List with Random Pointer

Description: Deep-copy a linked list where each node has a `random` pointer to any node or null.

Intuition: Interleave each clone directly after its original so a clone's `random` is reachable as `original.random.next` — no hash map needed. Three passes: weave clones in, wire random pointers, then unweave.

Algorithm: Pass 1: insert `clone` right after each `current`. Pass 2: set `current.next.random = current.random.next` (guarding null). Pass 3: detach the cloned chain and restore the original list.

Variables: `current` = walker over the list · `clone` = freshly made copy node · `sentinel` / `copy` = dummy + write pointer for extracting the cloned chain **Pseudocode:**

```
if list empty: return null
Pass 1: for each current, insert a fresh clone right after it (interleave)
Pass 2: for each original current, set clone.random = current.random's clone (current.random.next), guarc
Pass 3: walk again, detaching clones into their own list and restoring original next pointers
return head of the cloned list (sentinel.next)
```

```java
class Solution {
    public Node copyRandomList(Node head) {
        if (head == null) {
            return null;
        }
        // Pass 1: weave clones into the original list
        Node current = head;
        while (current != null) {
            Node clone = new Node(current.val);
            clone.next = current.next;
            current.next = clone;
            current = clone.next;
        }
        // Pass 2: assign random pointers on the clones
        current = head;
        while (current != null) {
            if (current.random != null) {
                current.next.random = current.random.next;
            }
            current = current.next.next;
        }
        // Pass 3: unweave into two separate lists
        Node sentinel = new Node(0);
        Node copy = sentinel;
        current = head;
        while (current != null) {
            copy.next = current.next;
            copy = copy.next;
            current.next = copy.next;   // restore original
            current = current.next;
        }
        return sentinel.next;
    }
}
```

Time O(n) | **Space** O(1)

#148 Sort List

Description: Sort a linked list. Aim for O(n log n) time and O(1) extra space (top-down recursion uses O(log n) stack).

Intuition: Merge sort fits linked lists perfectly — splitting is a pointer cut and merging reuses the #21 zipper. Find the middle, sort each half, merge.

Algorithm: Use fast/slow (with `fast = head.next` so the left half is never empty) to split the list in two. Recursively sort each half, then merge them with the standard sentinel-based merge.

Variables: `slow` / `fast` = splitting pointers (fast starts one ahead) · `second` = head of the right half · `left` / `right` = sorted halves · merge's `sentinel` / `current` = dummy + write pointer · `a` / `b` = merge inputs **Pseudocode:**

```
sortList(head):
    if 0 or 1 node: return head
    find middle with slow 1× and fast 2× (fast = head.next)
    second = slow.next; cut list into two halves
    left = sortList(head); right = sortList(second)
    return merge(left, right)
merge(a, b):
    make sentinel; current = sentinel
    while both exist: splice smaller head onto current; advance
    attach the leftover list
    return sentinel.next
```

```java
class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode second = slow.next;
        slow.next = null;                       // cut into two halves
        ListNode left = sortList(head);
        ListNode right = sortList(second);
        return merge(left, right);
    }
    private ListNode merge(ListNode a, ListNode b) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        while (a != null && b != null) {
            if (a.val <= b.val) {
                current.next = a;
                a = a.next;
            } else {
                current.next = b;
                b = b.next;
            }
            current = current.next;
        }
        current.next = (a != null) ? a : b;
        return sentinel.next;
    }
}
```

Time O(n log n) | **Space** O(log n)

#237 Delete Node in a Linked List

Description: Delete a node from a singly linked list given only a reference to that node (not the head). The node is guaranteed not to be the tail.

Intuition: Without access to the previous node you cannot unlink the given node directly, but you can copy the next node's value into it and then delete the next node instead.

Algorithm: Overwrite `node.val` with `node.next.val`, then bypass `node.next` by setting `node.next = node.next.next`.

Variables: `node` = the node to delete (only reference given; not the tail) **Pseudocode:**

```
copy the successor's value into node
unlink the successor (node.next = node.next.next)
```

```java
class Solution {
    public void deleteNode(ListNode node) {
        node.val = node.next.val;       // ← VARIATION: copy successor's value
        node.next = node.next.next;     // then unlink the successor
    }
}
```

Time O(1) | **Space** O(1)

#369 Plus One Linked List

Description: A non-negative integer is stored as a linked list of digits in big-endian order (most significant first). Add one to it and return the resulting list.

Intuition: Reverse the list so addition flows least-significant-first like normal carry arithmetic, add one with carry propagation, then reverse back. A possible new leading digit is handled by extending the tail after reversal.

Algorithm: Reverse the list. Walk it adding `carry` (starting at 1); each node becomes `(val + carry) % 10`, carry becomes `(val + carry) / 10`. If carry remains after the last node, append a new node. Reverse back to big-endian.

Variables: `reversed` = list reversed to least-significant-first · `carry` = carry digit (seeded with 1 = the +1) · `current` = walker · `last` = last visited node (for appending overflow) · `sum` = per-node total **Pseudocode:**

```
reverse the list (now least-significant first)
carry = 1
walk current while carry remains:
    sum = current.val + carry
    current.val = sum % 10; carry = sum / 10
    remember current as last; advance current
if carry left over: append a new node holding it after last
reverse back to big-endian and return
# reverse helper: standard previous/current/next reversal
```

```java
class Solution {
    public ListNode plusOne(ListNode head) {
        ListNode reversed = reverse(head);
        int carry = 1;                              // ← VARIATION: seed carry with the +1
        ListNode current = reversed, last = reversed;
        while (current != null && carry != 0) {
            int sum = current.val + carry;
            current.val = sum % 10;
            carry = sum / 10;
            last = current;
            current = current.next;
        }
        if (carry != 0) {
            last.next = new ListNode(carry);        // extend for a new leading digit
        }
        return reverse(reversed);
    }
    private ListNode reverse(ListNode head) {
        ListNode previous = null, current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        return previous;
    }
}
```

Time O(n) | **Space** O(1)

#430 Flatten a Multilevel Doubly Linked List

Description: Flatten a multilevel doubly linked list where nodes may have a `child` list. Produce a single-level list using the `next` pointers; all `child` pointers must be null.

Intuition: A child list should be spliced in immediately after its parent, before the parent's original `next`. A stack lets you remember each deferred `next` while you dive into a child branch — effectively an iterative DFS.

Algorithm: Walk `current` with a stack. When `current` has a child, push `current.next` (if non-null) for later, link `current` to the child, clear the child pointer, and fix `previous` / `next` doubly-linked wiring. When `current.next` is null but the stack is non-empty, pop and reattach.

Variables: `stack` = deferred continuations (original `next` pointers) · `current` = walker · `resumed` = continuation popped when a branch ends **Pseudocode:**

```
if head null: return null
stack empty; current = head
while current exists:
    if current has a child:
        push current.next (if any) to resume later
        splice the child in as current.next, fix prev pointer, clear child
    if current.next is null and stack not empty:
        pop a deferred continuation and reattach it (fix prev)
    advance current
return head
```

```java
class Solution {
    public Node flatten(Node head) {
        if (head == null) {
            return null;
        }
        Deque<Node> stack = new ArrayDeque<>();
        Node current = head;
        while (current != null) {
            if (current.child != null) {
                if (current.next != null) {
                    stack.push(current.next);   // defer the original continuation
                }
                current.next = current.child;
                current.next.prev = current;
                current.child = null;           // ← VARIATION: clear child after splicing
            }
            if (current.next == null && !stack.isEmpty()) {
                Node resumed = stack.pop();
                current.next = resumed;
                resumed.prev = current;
            }
            current = current.next;
        }
        return head;
    }
}
```

Time O(n) | **Space** O(n)

#445 Add Two Numbers II

Description: Two numbers are stored as linked lists in big-endian order (most significant digit first). Add them and return the sum as a linked list, also in big-endian order.

Intuition: Addition needs least-significant digits first, but reversing the inputs is discouraged here. Push all digits onto two stacks; popping yields least-significant-first. Build the result by prepending nodes so the final list is big-endian.

Algorithm: Push every digit of each list onto its own stack. Pop both stacks while either is non-empty or a carry remains, summing with carry. Prepend each new digit node to the front of the result so order is correct without a final reversal.

Variables: `stack1` / `stack2` = digit stacks (top = least significant) · `result` = result head built by prepending · `carry` = carry digit · `sum` = per-step total **Pseudocode:**

```
push all digits of l1 onto stack1, all of l2 onto stack2
result = null, carry = 0
while either stack non-empty or carry remains:
    sum = carry + popped digit of each non-empty stack
    carry = sum / 10
    prepend a new node holding sum % 10 to result
return result
```

```java
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        Deque<Integer> stack1 = new ArrayDeque<>();
        Deque<Integer> stack2 = new ArrayDeque<>();
        for (ListNode current = l1; current != null; current = current.next) {
            stack1.push(current.val);
        }
        for (ListNode current = l2; current != null; current = current.next) {
            stack2.push(current.val);
        }
        ListNode result = null;
        int carry = 0;
        while (!stack1.isEmpty() || !stack2.isEmpty() || carry != 0) {
            int sum = carry;
            if (!stack1.isEmpty()) {
                sum += stack1.pop();
            }
            if (!stack2.isEmpty()) {
                sum += stack2.pop();
            }
            carry = sum / 10;
            ListNode node = new ListNode(sum % 10);
            node.next = result;            // ← VARIATION: prepend to keep big-endian order
            result = node;
        }
        return result;
    }
}
```

Time O(m + n) | **Space** O(m + n)

#708 Insert into a Sorted Circular Linked List

Description: Insert a value into a sorted circular linked list and return a reference to any node in the list. The given pointer may reference any node, or be null for an empty list.

Intuition: Walk one full loop looking for the correct gap between a node and its successor. Three cases cover it: a normal in-between slot, the wrap-around boundary (max→min) where the value is an extreme, and a uniform-value list where any spot works.

Algorithm: If the list is empty, create a self-referential node. Otherwise scan `previous` / `current` around the ring. Insert when `previous.val <= val <= current.val`, or at the wrap point `previous.val > current.val` when `val` is `>= max` or `<= min`. If a full loop completes with no fit (all equal), insert anywhere.

Variables: `node` = new node to insert · `previous` / `current` = adjacent nodes scanned around the ring · `insertVal` = value to insert **Pseudocode:**

```
make node from insertVal
if list empty: make it self-referential and return it
previous = head, current = head.next
loop around the ring:
    if previous.val <= insertVal <= current.val: stop (normal slot)
    if at wrap point (previous.val > current.val) and insertVal is >= max or <= min: stop
    advance previous/current; if back to head (all equal): stop
splice node between previous and current
return head
```

```java
class Solution {
    public Node insert(Node head, int insertVal) {
        Node node = new Node(insertVal);
        if (head == null) {
            node.next = node;               // single-element circular list
            return node;
        }
        Node previous = head, current = head.next;
        while (true) {
            if (previous.val <= insertVal && insertVal <= current.val) {
                break;                                  // normal in-between slot
            }
            if (previous.val > current.val
                    && (insertVal >= previous.val || insertVal <= current.val)) {
                break;                                  // ← VARIATION: wrap-around boundary
            }
            previous = current;
            current = current.next;
            if (previous == head) {
                break;                                  // full loop: all values equal
            }
        }
        previous.next = node;
        node.next = current;
        return head;
    }
}
```

Time O(n) | **Space** O(1)

#725 Split Linked List in Parts

Description: Split a linked list into `k` consecutive parts as equal in size as possible. Earlier parts have sizes greater than or equal to later parts; some trailing parts may be empty.

Intuition: With length `n`, each part gets `n / k` nodes, and the first `n % k` parts get one extra. Walk through cutting off each part at its computed size.

Algorithm: Compute the length, then `baseSize = n / k` and `remainder = n % k`. For each of the `k` parts, the size is `baseSize + (i < remainder ? 1 : 0)`. Advance that many nodes, then sever the link to start the next part.

Variables: `length` = node count · `baseSize` = floor(length/k) per part · `remainder` = length mod k (extra nodes for early parts) · `result` = array of part heads · `current` = walker · `partSize` = size of the current part **Pseudocode:**

```
count length of list
baseSize = length / k; remainder = length % k
current = head
for i in 0..k-1:
    record current as result[i] (head of this part)
    partSize = baseSize + (1 if i < remainder else 0)
    walk current to the last node of this part
    if current exists: save next, sever this part, move to next
return result
```

```java
class Solution {
    public ListNode[] splitListToParts(ListNode head, int k) {
        int length = 0;
        for (ListNode current = head; current != null; current = current.next) {
            length++;
        }
        int baseSize = length / k;
        int remainder = length % k;                 // ← VARIATION: first `remainder` parts get +1
        ListNode[] result = new ListNode[k];
        ListNode current = head;
        for (int i = 0; i < k; i++) {
            result[i] = current;
            int partSize = baseSize + (i < remainder ? 1 : 0);
            for (int j = 0; j < partSize - 1; j++) {
                current = current.next;
            }
            if (current != null) {
                ListNode next = current.next;
                current.next = null;                // sever this part
                current = next;
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(k)

#1171 Remove Zero Sum Consecutive Nodes from Linked List

Description: Repeatedly remove consecutive sequences of nodes that sum to zero until no such sequence remains. Return the resulting list.

Intuition: Use prefix sums. If two nodes share the same running prefix sum, every node strictly between them sums to zero and can be dropped. A hash map from prefix sum to node lets you splice out those runs in one pass.

Algorithm: First pass: compute prefix sums over a sentinel-prefixed list, storing the last node seen for each prefix sum in a map (later occurrences overwrite earlier ones). Second pass: for each prefix sum, jump directly to the last node holding that same sum, skipping the zero-sum span.

Variables: `sentinel` = dummy before head · `lastSeen` = map prefix-sum → last node with that sum · `prefix` = running prefix sum · `current` = walker **Pseudocode:**

```
make sentinel pointing at head
Pass 1: walk from sentinel, accumulate prefix sum, store lastSeen[prefix] = current (overwrite)
Pass 2: reset prefix; walk from sentinel:
    accumulate prefix
    rewire current.next to lastSeen[prefix].next (skip any zero-sum span)
return sentinel.next
```

```java
class Solution {
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        Map<Integer, ListNode> lastSeen = new HashMap<>();
        int prefix = 0;
        for (ListNode current = sentinel; current != null; current = current.next) {
            prefix += current.val;
            lastSeen.put(prefix, current);          // ← VARIATION: keep the latest node per sum
        }
        prefix = 0;
        for (ListNode current = sentinel; current != null; current = current.next) {
            prefix += current.val;
            current.next = lastSeen.get(prefix).next;   // skip any zero-sum span
        }
        return sentinel.next;
    }
}
```

Time O(n) | **Space** O(n)

#1265 Print Immutable Linked List in Reverse

Description: Print all values of an immutable linked list in reverse, using only the exposed `getNext()` and `printValue()` API, without modifying the list.

Intuition: Recursion naturally reverses order — recurse to the end first, then print on the way back up the call stack.

Algorithm: If the node is null, return. Otherwise recurse on `head.getNext()` first, then call `head.printValue()`. Values print from tail to head as the stack unwinds.

Variables: `head` = current node in the recursion (uses only getNext / printValue) **Pseudocode:**

```
printReverse(head):
    if head is null: return
    printReverse(head.getNext())   # recurse to the end first
    head.printValue()              # print on the way back up
```

```java
class Solution {
    public void printLinkedListInReverse(ImmutableListNode head) {
        if (head == null) {
            return;
        }
        printLinkedListInReverse(head.getNext());   // ← VARIATION: recurse first, print on unwind
        head.printValue();
    }
}
```

Time O(n) | **Space** O(n)